

## 16.1. Контрольні запитання

### 16.1.1. Для чого застосовується шаблон проєктування Ітератор?

**Ітератор** — це поведінковий шаблон проєктування, який вирішує наступні завдання:

- **Абстракція обходу:** дозволяє послідовно перебирати елементи складових об'єктів (колекцій), не розкриваючи їхню внутрішню структуру (реалізацію). Клієнту не важливо, як організовані дані: у вигляді списку, дерева, хеш-таблиці чи графа.
- **Єдиний інтерфейс:** надає стандартний спосіб навігації по різних типах колекцій, що робить клієнтський код універсальним.
- **Підтримка декількох стратегій обходу:** дозволяє одночасно мати кілька різних варіантів перебору елементів (наприклад, пошук у глибину чи в ширину, за алфавітом чи за ціною).
- **Спрощення колекції:** знімає з класу колекції обов'язок реалізації методів обходу, зосереджуючи його лише на збереженні даних (дотримання принципу Single Responsibility).

### 16.1.2. Наведіть основні етапи реалізації шаблону Ітератор.

Реалізація шаблону зазвичай складається з таких кроків:

1. **Визначення інтерфейсу Ітератора:** Створюється інтерфейс, який описує методи для перевірки наявності наступного елемента та отримання самого елемента (наприклад, стандартні для Java `hasNext()` та `next()`).
2. **Створення конкретного Ітератора:** Реалізується клас, який інкапсулює логіку обходу конкретної колекції. Він зберігає поточну позицію та посилання на колекцію, яку перебирає.
3. **Визначення інтерфейсу Колекції:** Створюється інтерфейс (наприклад, `Iterable` у Java), який оголошує метод для отримання об'єкта ітератора (наприклад, `iterator()`).
4. **Реалізація конкретної Колекції:** Клас колекції реалізує метод створення ітератора, повертаючи екземпляр відповідного конкретного ітератора, передаючи йому необхідні дані для роботи.
5. **Використання клієнтом:** Клієнт отримує ітератор від колекції та використовує його методи в циклі, не звертаючись до структури самої колекції напряму.

